**NYC Citi Bike 2019 Data Analysis & BI Dashboard**

## 1. Project Phases & Methodology

To transform 20.5 million raw trip records into actionable business insights, this project followed a structured, end-to-end data lifecycle divided into five distinct phases:

### Phase 1: Data Ingestion & ETL Pipeline (Python)

- **Extraction:** Automated the download and extraction of 12 monthly CSV files (2019) from the AWS S3 bucket.
- Bronze Layer (RawDataLoader): Ingested 12 monthly CSVs into a single consolidated raw Parquet file. Used Polars' scan_csv and sink_parquet for lazy evaluation and memory-efficient streaming, allowing us to process 20M+ rows on standard hardware without Out-Of-Memory (OOM) errors.
- Silver Layer (DataTransformer): Strictly separated from the loading phase. This module applied feature engineering (Haversine distance, time/calendar attributes), mapped string categories to integer IDs, and enforced rigorous data quality filters.
- Gold Layer (Consumption-Ready Star Schema): The final physical handoff to the BI tool. The cleaned Silver data was normalized into a Star Schema and exported as highly optimized, Zstandard-compressed Parquet files into the data/processed/ directory.
    - Fact Table (fact_trips_clean): Kept intentionally lean, containing only foreign keys, timestamps, and numerical metrics to ensure fast aggregations.
    - Dimension Tables (dim_station, dim_user_type, dim_gender): Exported as separate, deduplicated lookup tables to enforce referential integrity and reduce data redundancy.

### Phase 2: Data Profiling & Quality Assurance

- Production-Grade Logging: Implemented a custom LoggerManager featuring dual-output logging. Console logs use a human-readable format for development, while file logs use a structured JSON format via a RotatingFileHandler for production monitoring and debugging.
- Conducted a deep-dive exploratory analysis on the Bronze dataset to identify anomalies.
- Documented critical data quality issues, including missing station coordinates, impossible trip durations (e.g., trips lasting >24 hours), demographic outliers (e.g.,

birth year 1857), and coordinate drift (60 station IDs mapping to multiple GPS locations).

**Phase 3: Data Modeling for Business Intelligence**

- Restructured the flat Silver dataset into a normalized **Star Schema** optimized for Power BI.
- Created a central Fact table (fact_trips_clean) containing metrics and foreign keys.
- Built dedicated Dimension tables for Dates (dim_date), Stations (dim_station), User Types (dim_user_type), and Gender (dim_gender).
- Implemented role-playing dimension logic to handle both Start and End station/time relationships without duplicating tables.

**Phase 4: Exploratory Data Analysis (Python)**

- Leveraged Polars and Matplotlib/Seaborn to answer complex, computation-heavy business questions.
- Analyzed demographic reliability between user types (Q1), identified the most frequent trip routes (Q3), and investigated average bike speeds using the Haversine distance metric (Q6).

**Phase 5: Interactive Visualization & Reporting (Power BI)**

- Imported the Star Schema into Power BI and developed DAX measures for time intelligence and complex filtering.
- Applied the official **Henkel corporate color palette** to ensure brand consistency and visual accessibility.
- Built interactive dashboards to analyze hourly/weekly usage patterns (Q2), trip duration distributions (Q4), and subscriber commute tidal flows (Q5).
- Designed a dedicated "Executive Insights" page to summarize key findings and strategic recommendations for stakeholders.

**2. Key Business Findings**

The analysis of 20.5 million Citi Bike trips reveals two distinctly different user personas: **Subscribers** (annual members acting as daily commuters) and **Customers** (short-term pass users acting as tourists and leisure riders).

**A. Distinct User Personas and Route Preferences**

- **Demographics & Data Reliability:** Subscribers are generally older (median age 34–36) compared to Customers (median age 29). Furthermore, Customer demographic data is highly unreliable for gender analysis, with **43.2%** marked as "Unknown," compared to only **1.81%** for Subscribers.
- **Route Behavior:** Subscribers utilize highly predictable, point-to-point commuter routes (e.g., *E 7 St & Avenue A ➔ Cooper Square*). In contrast, Customers favor recreational loops and tourist landmarks (e.g., *Central Park South*, *Soissons Landing*), often returning to their starting dock.

**B. Temporal Dynamics and "Tidal" Commute Flows**

- **Hourly Patterns:** Subscribers exhibit a classic "M-shaped" usage curve with sharp peaks at **8:00 AM** and **5:00 PM**. Customers show a smooth "bell curve," peaking during midday leisure hours (12:00 PM – 3:00 PM).
- **Weekly Patterns:** Subscriber usage drops significantly on weekends, while Customer usage spikes, confirming their recreational profile.
- **The Tidal Flow (Rentals vs. Returns):** For top commuter stations on weekdays, Subscriber Rentals peak at **8:00 AM**, with Returns following at **9:00 AM** (a 1-hour commute lag). This pattern perfectly inverts in the evening, with Rentals peaking at **5:00 PM** and Returns at **6:00 PM**.

**C. Trip Duration and Speed Metrics**

- **Duration Distribution:** There is a massive divergence in trip lengths. **94.86%** of Subscriber trips last **0–10 minutes**, reflecting efficient commutes. Conversely, **72.59%** of Customer trips last **over 60 minutes**.
- **Average Speed:** Subscribers ride significantly faster, averaging **9.19 km/h**, while Customers average **6.23 km/h**. Subscriber speeds peak during rush hours, while Customer speeds dip during midday, likely due to navigating crowded tourist areas.

**D. Strategic Business Recommendations** Based on these findings, the following operational strategies are recommended:

1. **Fleet Rebalancing:** Shift from static distribution to dynamic, time-based rebalancing. Move bikes from residential to business zones between 6:00 AM – 9:00 AM, and reverse the flow between 4:00 PM – 7:00 PM.
2. **Infrastructure & Safety:** The slower, variable speeds of Customers in tourist zones suggest high friction. Prioritize protected bike lanes and clearer signage around landmarks like Central Park to prevent pedestrian conflicts.

3. **Targeted Marketing & Pricing:** Promote annual memberships to daily commuters by highlighting rush-hour efficiency. For Customers, introduce premium "hourly passes" or time-based pricing to monetize their tendency to exceed the standard 30-minute ride window.

**3. Data Quality Issues, Assumptions, and Cleaning Decisions**

To ensure the accuracy and reliability of the final dashboard, a rigorous data quality framework was applied during the ETL pipeline.

**A. Main Data Quality Issues Identified**

- **Impossible Trip Durations:** 6,118 trips (0.03%) had durations exceeding 24 hours, with a maximum of 44 days. These represent lost bikes, forgotten returns, or system glitches.
- **Demographic Outliers:** 9,969 records (0.05%) had birth years prior to 1920 (e.g., 1857), implying riders over 100 years old, which are clear data entry errors.
- **Coordinate Drift (Duplicate Primary Keys):** Profiling revealed that 60 unique station IDs mapped to multiple different latitude/longitude combinations. This occurs when stations physically relocate or suffer from GPS coordinate errors.
- **Missing Spatial Data:** Exactly 180 rows (0.001%) had null values for station IDs, names, and coordinates.
- **Incomplete Demographics:** While not strictly "missing" (encoded as 0), 43.2% of Customer trips had an "Unknown" gender, compared to only 1.81% for Subscribers.

**B. Cleaning Decisions Implemented in the Pipeline**

- **Duration & Speed Filtering:** Trips lasting > 24 hours (86,400 seconds) were dropped. Additionally, calculated speeds exceeding 40 km/h were filtered out to remove GPS coordinate jumps.
- **Age Capping:** Birth years were used to calculate age, which was then strictly capped between 12 and 90 years old. Records falling outside this range were set to Null for demographic analysis.
- **Resolving Coordinate Drift:** To maintain a clean 1-to-Many relationship in the Power BI Star Schema, the dim_station table was deduplicated. For the 60 drifting stations, we retained only the latitude/longitude pair that appeared most frequently in the dataset.

- **Null Filtering:** The 180 rows with missing spatial data were completely removed from the Fact table to prevent broken joins in the BI layer.

**C. Key Analytical Assumptions**

- **Haversine Distance as a Proxy:** We used the Haversine formula to calculate the straight-line distance between start and end stations. We assume this is a valid proxy for actual riding distance. While it slightly underestimates the true distance (as bikes must follow streets), it provides a consistent, mathematically sound baseline for calculating average speed across 20 million trips.
- **Gender Encoding:** We assume that a gender value of 0 explicitly means the user "Opted Out/Unknown" during checkout, rather than a system failure, particularly for short-term Customer transactions.
- **Most Frequent Coordinate = True Location:** For stations with coordinate drift, we assume the most frequently recorded GPS coordinate represents the station's primary or most accurate physical location.

**4. Data Structuring for Reporting (Star Schema & Relationships)**

To ensure high performance and ease of use in Power BI, the data was heavily transformed in Python before being loaded into the BI tool. We transitioned from a flat, denormalized dataset into a highly optimized **Star Schema**.

**A. Prepared Tables (The Gold Layer)** The data was split into a central Fact table surrounded by Dimension tables, all exported as Zstandard-compressed Parquet files:

- **Fact Table (fact_trips_clean):** Kept intentionally lean to ensure fast aggregations. It contains only numerical metrics (trip_duration_seconds, distance_km, speed_kmh) and Foreign Keys (user_type_id, gender_id, start_station_id).
- **Dimension Tables:**
    - dim_station: Contains unique station IDs, names, and coordinates.
    - dim_user_type: Maps IDs to "Subscriber" and "Customer".
    - dim_gender: Maps IDs to "Unknown", "Male", and "Female".
    - dim_date: A continuous calendar table generated to handle time-intelligence functions.

**B. Data Relationships** The tables were linked using strict **One-to-Many (1:*)** relationships with **Single-directional** cross-filtering. This is a critical best practice that prevents ambiguous filter paths and ensures DAX measures calculate correctly.

- **Role-Playing Dimensions (Inactive Relationships):** To analyze both the start and end of a trip without duplicating dimension tables, we implemented inactive relationships. For example, dim_station is actively linked to start_station_id, but an inactive relationship is maintained with end_station_id. In Power BI, DAX functions like USERELATIONSHIP() are used to activate the "End Station" context dynamically when needed.

**C. Transformations Performed Before Loading into Power BI** By performing heavy transformations in the Python ETL pipeline rather than in Power BI, we significantly reduced the memory footprint and refresh time of the dashboard:

- **Spatial & Speed Calculations:** The Haversine distance (straight-line distance between coordinates) and average speed (speed_kmh) were calculated in Python.
- **Categorical Mapping:** String columns (like "Customer" / "Subscriber") were mapped to integer IDs in the Fact table to save memory, while the descriptive strings were moved to the Dimension tables.
- **Temporal Parsing:** Raw string timestamps were parsed into proper DateTime objects, and calendar attributes (Year, Month, Day of Week) were extracted.
- **Dimension Deduplication:** The 60 station IDs suffering from "coordinate drift" were resolved in Python by grouping by station_id and retaining the most frequent latitude/longitude pair, ensuring a clean Primary Key for Power BI.

**5. Automation, Refresh, and Scaling Strategy**

While the current solution operates locally as a robust prototype, transitioning this into a recurring, enterprise-grade reporting solution requires a strategic approach to automation, data refresh, and scalability.

**A. Automating the ETL Pipeline**

- **Orchestration:** The modular Python pipeline (src/main.py) can be containerized using Docker and orchestrated via tools like **Apache Airflow**, or **Azure Data Factory**.
- **Trigger Mechanism:** The pipeline can be scheduled to run automatically on the 15th of every month (when Citi Bike releases new data) or configured as an event-

driven trigger that activates immediately when a new CSV is dropped into a cloud storage bucket.

- **Monitoring:** The existing JSON logging structure (RotatingFileHandler) can be integrated with monitoring tools like Datadog to send instant Slack/Teams alerts if the pipeline fails or if data quality thresholds are breached.

**B. Refreshing the Power BI Dashboard**

- **Cloud Storage Integration:** Instead of reading from a local data/processed/ folder, the Gold Layer Parquet files would be pushed to a cloud service (e.g., Databricks or Snowflake).
- **Scheduled Refresh:** Power BI Service would connect directly to the cloud storage via native connectors. A **Scheduled Refresh** would be configured (e.g., daily at 6:00 AM) to automatically pull the latest Parquet files.

**C. Scaling for Future Growth**

- **Compute Scaling (Polars to Spark):** The current pipeline leverages Polars' lazy evaluation and streaming engine, which is highly memory-efficient and can handle up to ~100M rows on a single machine. If data volume exceeds this, the src/transform/ logic can be seamlessly migrated to **PySpark on Databricks** or **Snowflake**, as the underlying Medallion architecture (Bronze/Silver/Gold) remains exactly the same.
- **Partitioning:** The Gold Layer Parquet files would be partitioned by Year and Month (e.g., .../fact_trips/year=2019/month=10/). This allows Power BI and the ETL pipeline to perform "partition pruning," scanning only the specific folders needed for a query, drastically improving performance.

**6. Proposed Additional Data for Deeper Insights**

While the 2019 Citi Bike dataset provides a robust foundation for analyzing rider behavior and temporal trends, integrating supplementary data sources would significantly elevate the analytical depth and operational value of the solution.

**A. Hyper-Local Weather Data (Temperature, Precipitation, Wind Speed)**

- **Rationale:** Bike-sharing is inherently weather-dependent. By joining hourly historical weather data (e.g., from NOAA or OpenWeatherMap) to our dim_date and fact_trips tables, we could isolate the exact impact of rain or extreme temperatures on ridership. This would allow us to differentiate between a drop in usage caused by a seasonal trend versus a drop caused by a stormy week.

**B. Real-Time Dock Inventory and Station Capacity**

- **Rationale:** Our current analysis assumes that a drop in trips at a specific hour is due to low demand. However, it could actually be due to *low supply* (e.g., the station was completely empty, or the destination station had no empty docks to accept returns). Ingesting historical dock inventory data would allow us to calculate a "Lost Demand" metric, identifying how many potential trips were abandoned due to operational bottlenecks.

**C. Bike Type/Model (E-bike vs. Classic Pedal)**

- **Rationale:** In our Python analysis (Question 6), we noted that Subscribers ride significantly faster than Customers (9.19 km/h vs. 6.23 km/h). While this is largely due to commuter vs. tourist behavior, it is also heavily influenced by bike type. Knowing whether a trip was taken on a standard bike or an e-bike would allow us to normalize the speed analysis and track the adoption and impact of the e-bike fleet expansion.